
Neural Network Framework from Scratch

January 12, 2026

Valerio Melissa¹

Abstract

This project presents an implementation of a neural network framework built entirely from scratch in CUDA C++ with dual CPU and GPU implementations. Currently the framework allows the creation of a multilayer perceptron with configurable depth, activation functions, and loss functions. To prove correctness, the framework is tested on the MNIST digit-recognition dataset. Code available at: <https://github.com/Valerio3240264/nn-scratch>.

1. Introduction

This project originated from a desire for greater low-level control than Python typically offers. Rather than adopting an existing C++ framework, I chose to build a neural network framework from scratch. The result is not comparable to established frameworks like PyTorch or TensorFlow, but it has been a great learning experience, especially about CUDA kernels, weight initialization, and backpropagation.

The main characteristic of the project is the dual implementation of the framework, one for the CPU and one for the GPU, selectable when declaring the MLP.

2. Related Work

Backpropagation. Classic backpropagation (Rumelhart et al., 1986).

Xavier/Glorot initialization. Variance-preserving init for tanh/linear (Glorot & Bengio, 2010).

He initialization. Variance-preserving init for ReLU (He et al., 2015).

Activations and losses. The MLP supports ReLU, Tanh, Linear, and softmax, paired with MSE for regression and cross-entropy for classification.

¹**AUTHORERR: Missing \dlaiaffiliation.** Email: Valerio Melissa <melissa.2139153@studenti.uniroma1.it>.

Stochastic Gradient Descent. Updates use plain SGD (vectorized CUDA update kernel on GPU, straightforward loop on CPU), keeping the focus on correctness of gradients rather than optimizer features.

3. Method

3.1. Core Components

- **Input.** Holds the layer input and partial gradients.
- **Weight.** Stores weights and biases with Xavier/Glorot or He initialization; performs the forward pass $y = Wx + b$ and accumulates gradients during backpropagation.
- **Activation.** Applies the chosen activation function (ReLU, Tanh, Linear and Softmax) on the weight outputs.
- **Layer.** Connects input, weight, and activation components and executes forward/backward passes.
- **MLP.** Initializes layers and loss, links components, and orchestrates full forward/backward passes.
- **Loss.** Computes the loss and starts backward propagation to the first layer.

3.2. Computational Graph Architecture

The framework implements a directed acyclic graph (DAG) where nodes represent computations and edges represent data flow.

Layer $\longrightarrow$ Layer $\text{----} \Rightarrow$ Layer $\longrightarrow$ Loss

Figure 1. Graph representation of the MLP.

Layer internals follow a similar pattern, coupling inputs, weights, and activations.

Input $\longrightarrow$ Weights $\longrightarrow$ Activation

Figure 2. Layer-level computational graph.

3.3. Forward Pass

During the forward pass, data flows sequentially through each layer of the network. Each layer computes the linear transformation $y = Wx + b$ using its weights W and biases b on the incoming activation x . The result y is then passed through the chosen activation function (ReLU, Tanh, Linear, or Softmax), generating an output that serves as the input for the next layer. This process continues until the output of the final layer is produced. The loss component then store the result waiting for a target output to compute the loss and start the backward pass. In the code, calling `mlp->operator()(input)` triggers this cascade of computations, storing intermediate results in each component for use during backpropagation.

3.4. Backward Pass

The backward pass (backpropagation (Rumelhart et al., 1986)) computes gradients via the chain rule from loss to inputs. Each layer applies activation derivatives, then multiplies by W^T to propagate gradients and accumulates weight/bias gradients. Finally, SGD updates parameters. Calling `mlp.compute_loss(target)` runs loss + full backward pass for one sample.

3.5. GPU Implementation

CUDA kernels use vectorization, tiling, shared memory, reductions, and coalescing for matmul/softmax speedups.

Kernels implemented are:

- `activation.cu`: forward/backward for ReLU, Tanh, Linear, Softmax.
- `loss.cu`: cross-entropy and MSE plus gradients.
- `matrix.cu`: mat-vec, backward, SGD updates.
- `softmax.cu`: stable softmax forward/backward.
- `utils.cuh`: helpers for launches, indexing, device memory.

4. Experimental Results

4.1. MNIST Training (CPU vs GPU)

To evaluate correctness, two tests train the same MLP on MNIST: a CPU run and a GPU run. The model uses three layers with 256, 128, and 10 neurons, ReLU activations for the first two layers, softmax for the last layer, and cross-entropy loss. Training uses 5 epochs, batch size 100, 32k training samples, and 10k validation samples.

Table 1. Performance comparison.

Device	Epochs	Batch Size	Valid. Accuracy	Time/epoch	Total Time
CPU	5	100	96.90	18.64	93.18
GPU	5	100	96.83	5.48	27.38

5. Discussion and Conclusions

As the project is a learning exercise, there are some weaknesses and future work that can boost the performance and usability such as:

- Batch processing: the current implementation doesn't support the batch processing even though the logic keep being the same as now but it requires additional time and was not the primary goal of the project.
- Missing different optimizers: the current project doesn't have an optimizer class, the parameters update is managed by the `mlp` that calls the layers and each one calls the weights component that has a method that updates the weights: $W^i = W^{i-1} - \eta \nabla W^{i-1}$

The GPU version is significantly faster than the CPU version while delivering comparable accuracy. Results will vary with hardware and system configuration but consistently reach around 95% validation accuracy, supporting the correctness of the implementation. The project has been a great learning experience and will be updated in the future.

References

- Glorot, X. and Bengio, Y. Understanding the difficulty of training deep feedforward neural networks. 9:249–256, 2010. URL <http://proceedings.mlr.press/v9/glorot10a.html>.
- He, K., Zhang, X., Ren, S., and Sun, J. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. *2015 IEEE International Conference on Computer Vision (ICCV)*, pp. 1026–1034, 2015. doi: 10.1109/ICCV.2015.123. URL <https://doi.org/10.1109/ICCV.2015.123>.
- Rumelhart, D. E., Hinton, G. E., and Williams, R. J. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986. doi: 10.1038/323533a0. URL <https://doi.org/10.1038/323533a0>.